

```
$ tino --docs
```

PROJECT DOCUMENTATION

TINO

A terminal-based, Chrome-Dino-style endless runner for macOS and Linux — built in pure Python.

Document set	Overview, PRD, Architecture, Game Design, Terminal Behavior, CLI, Data Model, Testing, Tech Stack, Roadmap
Platform	macOS & Linux · Bash & Zsh
Stack	Python 3.12+, curses, argparse, JSON, pytest, Ruff
Version	v1 Planning Document

STATUS: PLANNING COMPLETE

About This Documentation

TINO is a terminal-based endless runner in the spirit of Chrome's offline dinosaur game, launched with a single `tino` command. This document set is the single source of truth for what TINO is, how it is built, and why — covering product requirements, system architecture, game design, terminal behavior, CLI, data model, testing strategy, tech stack, and the release roadmap.

Read the sections in order if you're new to the project; each is self-contained but cross-references related decisions elsewhere in the set.

#	Section	What's in it
1	Overview	What TINO is, who it's for, product goals
2	Requirements (PRD)	Functional & non-functional requirements, v1 scope
3	Architecture	Module breakdown, project structure, key design decisions
4	Game Design	Mechanics, controls, scoring, difficulty curve
5	Terminal Behavior	curses usage, cross-shell/OS handling, resize/input
6	CLI	Commands and flags
7	Data Model	Config & high-score schemas, storage design
8	Testing	Test strategy, structure, CI
9	Tech Stack	Every tool used, and why
10	Roadmap	v1 → v2 → v3 plan

Status: Planning complete. No application code has been written yet — implementation starts only once the architecture and docs below are approved.

Table of Contents

10 sections

01	Overview
02	Requirements (PRD)
03	Architecture
04	Game Design
05	Terminal Behavior
06	CLI
07	Data Model
08	Testing
09	Tech Stack
10	Roadmap

01 Overview

What TINO is

TINO is a terminal-based endless runner in the spirit of Chrome's offline dinosaur game. You run it with:

```
tino
```

The player character auto-runs across the terminal and must jump over incoming obstacles. The game speeds up over time; the run ends the first time an obstacle is hit. The goal each run is to beat your own high score.

Why terminal-based

No GUI framework, no browser, no external runtime — just Python and the terminal. This keeps the project:

- **Portable:** runs anywhere Python + a terminal exists (macOS, Linux).
- **Small in surface area:** the whole dependency footprint is the Python standard library plus dev tooling (pytest, Ruff). Nothing to compile, no rendering engine to fight.
- **A clean showcase of core CS/software-engineering fundamentals** — a real game loop, state management, and terminal I/O, without a framework doing the hard parts for you.

Who it's for

- Primary: developers who live in a terminal and want a quick, satisfying break without leaving it.
- Secondary (project goal): a polished, real piece of software that demonstrates practical system design — not a toy script.

Product goals

1. Smooth, responsive gameplay in a terminal (no visible flicker/lag on a normal machine).
2. Works identically whether launched from Bash or Zsh, on macOS or Linux.
3. High scores persist locally between runs.
4. Codebase stays maintainable as features are added — game logic isolated from terminal-specific code so either could change independently.
5. Installable and runnable by someone who isn't the author, with minimal setup friction.

Non-goals (for now)

- No GUI/web version.

- No networked multiplayer.
- No Windows support in v1 (curses' Windows story is inconsistent; revisit later if needed).

02 Requirements (PRD)

2.1 Functional requirements — v1

ID	Requirement
FR-1	Running <code>tino</code> launches the game directly into a start screen.
FR-2	Player can start a run, jump over obstacles, and the run ends on collision.
FR-3	A single obstacle type exists in v1 (ground-level, e.g. cactus-style block).
FR-4	Game speed increases progressively the longer a run lasts.
FR-5	Score increases automatically for as long as the player survives.
FR-6	On game over, the current run's score is compared against the stored high score; if higher, it's saved.
FR-7	Player can restart a run without relaunching the program (e.g. press a key at the game-over screen).
FR-8	Player can quit cleanly at any point (e.g. <code>q</code> or <code>Ctrl+C</code>), and the terminal is restored to its normal state on exit.
FR-9	CLI supports at least: running the game, viewing the current high score, and resetting the high score.
FR-10	The game behaves identically when launched from Bash or Zsh, on macOS or Linux.

2.2 Non-functional requirements

ID	Requirement
NFR-1	Performance: consistent frame pacing — no perceptible stutter on a typical laptop.
NFR-2	Resilience: an unhandled error must not leave the user's terminal in a broken/raw state.
NFR-3	Portability: standard library + the listed dependencies only; no OS-specific code paths beyond what <code>curses</code> already abstracts.
NFR-4	Maintainability: game logic (rules, state, scoring) must be testable without a real terminal attached.
NFR-5	Test coverage: core engine logic (collision, scoring, difficulty, state transitions) covered by unit tests; UI layer covered where practical.
NFR-6	Config: sensible defaults with no required setup; an optional config file can override behavior (e.g. key bindings) later.

2.3 In scope for v1

- Single-player, single obstacle type, progressive difficulty, local high score, start/playing/game-over flow, CLI with run/show-score/reset-score.

2.4 Explicitly out of scope for v1 (see [Roadmap](#))

- Multiple obstacle types / obstacle variety
- Duck/crouch mechanic
- Sound
- Themes/skins
- Cloud-synced high scores
- PyPI packaging (planned, not v1)
- Windows support

2.5 Success criteria

- A new user can clone the repo, follow the README, and be playing within a couple of minutes.
- The game runs without visual glitches on both macOS Terminal/iTerm2 and common Linux terminal emulators.
- Core engine test suite passes in CI on every push.

03 Architecture

3.1 Guiding principle

Game logic (rules, state, scoring, collision) must not know that a terminal exists. The terminal/curses layer is just one possible "renderer + input source" sitting on top of a plain Python engine. This is what makes the engine unit-testable without a real terminal, and lets the rendering layer change later without touching game rules.

3.2 Project structure

```
тино/
├─ src/тино/
│  ├─ __init__.py
│  ├─ cli.py           # argparse entry point – parses args, dispatches to a command
│  ├─ app.py           # wires everything together: config → engine → UI → run loop
│  ├─ config.py        # loads defaults, merges optional user config
│  └─ engine/
│     ├─ __init__.py
│     ├─ game.py       # Game class: owns the state machine + drives the game loop
│     ├─ state.py      # GameState enum + transition rules (MENU, PLAYING, GAME_OVER)
│     ├─ entities.py   # Player, Obstacle – includes jump/gravity movement logic
│     ├─ collision.py   # pure collision-detection functions
│     └─ difficulty.py  # speed/spawn-rate progression as a function of elapsed time
│  └─ ui/
│     ├─ __init__.py
│     ├─ terminal.py   # curses init/teardown, screen size, resize handling
│     ├─ renderer.py   # draws engine state to the screen – no game logic here
│     └─ input.py      # non-blocking key reads, maps keys → engine actions
│  └─ storage/
│     ├─ __init__.py
│     ├─ highscore.py  # HighScoreStore interface + JSONHighScoreStore
│     └─ paths.py      # resolves the OS-appropriate data directory
├─ tests/
│  ├─ unit/            # engine, storage, config – no terminal required
│  └─ integration/     # thin checks that the pieces wire together correctly
├─ scripts/
│  ├─ setup.sh         # create venv, install deps
│  ├─ lint.sh          # run Ruff
│  └─ test.sh          # run pytest
├─ docs/              # this folder
├─ .github/workflows/ci.yml
├─ pyproject.toml
└─ README.md
```

3.3 Module responsibilities

Module	Owns	Does NOT know about
engine/game.py	The state machine, the game loop tick, coordinating entities/collision/difficulty	curses, terminal size, keycodes

Module	Owns	Does NOT know about
<code>engine/entities.py</code>	Player and Obstacle position/movement (including jump arc — see 3.5)	Rendering, input
<code>engine/collision.py</code>	Whether two entities overlap	Everything else — pure functions in, booleans out
<code>engine/difficulty.py</code>	Mapping elapsed time → current speed	Rendering, storage
<code>ui/terminal.py</code>	Starting/stopping curses safely, terminal size	Game rules
<code>ui/renderer.py</code>	Drawing a given engine state to the screen	Game rules — it only reads state, never mutates it
<code>ui/input.py</code>	Reading a keypress and mapping it to an action (<code>JUMP</code> , <code>QUIT</code> , <code>RESTART</code>)	What the action does
<code>storage/highscore.py</code>	Reading/writing the high score	The engine — the engine calls it, not the reverse
<code>cli.py</code>	Parsing <code>tino</code> command-line arguments	Game internals — it just starts <code>app.py</code> with the right options

3.4 The game loop

A single fixed-interval loop, driven from `app.py` :

1. Read input (non-blocking) → translate to an action.
2. Advance engine state by one tick (move entities, check collisions, update difficulty/score).
3. Render the current state.
4. Sleep for the remainder of the frame interval.

This is a standard **game loop** pattern — chosen because it's the simplest correct way to get consistent pacing in a terminal, and every other design (event-driven redraw, for instance) adds complexity without benefit for a single-screen game like this.

3.5 Why no separate "Physics" module

A full physics module implies general simulation (multiple bodies, forces, constraints). TINO has exactly one moving vertical trajectory: the player's jump (a fixed initial velocity plus constant gravity, each tick). That's a few lines of arithmetic, not a subsystem — so it lives directly in `entities.py` next to the `Player` it belongs to. If TINO ever needs real physics (it won't for a dino runner), this is the seam where it would be extracted.

3.6 State machine

```
MENU —(start)—> PLAYING —(collision)—> GAME_OVER
^                                     |
|_____ (restart) _____|
```

Justified because the game genuinely has distinct modes with different valid inputs and different rendering — a raw boolean ("is playing?") wouldn't cleanly express "just game-over, waiting for restart" vs "actively playing." `engine/state.py` defines the enum and the only legal transitions between them; `game.py` is the sole place that triggers a transition.

3.7 Storage design (for future cloud sync)

`storage/highscore.py` defines a small interface:

```
class HighScoreStore(Protocol):
    def load(self) -> int: ...
    def save(self, score: int) -> None: ...
```

`JSONHighScoreStore` is the only implementation in v1, backed by a JSON file under a per-user data directory (see [Data Model](#)). The engine depends only on the interface. If a remote/cloud store is added later, it's a new class implementing the same two methods — no changes needed anywhere else. This is intentionally the smallest possible abstraction that satisfies "local now, swappable later" — not a general plugin framework.

3.8 Error handling

- Terminal setup/teardown (`ui/terminal.py`) uses `try/finally` so the terminal is always restored to a normal state, even on an unhandled exception or `Ctrl+C`.
- Storage reads/writes are wrapped narrowly: a missing or corrupt high-score file is treated as "no high score yet" rather than crashing the game.
- The CLI validates arguments up front (via `argparse`) and fails with a clear message rather than surfacing a stack trace.

3.9 Configuration

`config.py` defines defaults in code (frame rate, initial speed, key bindings) and optionally merges a user config file if one exists (see [Data Model](#)). No config file is required to run the game — this keeps FR (no required setup) satisfied while leaving room to customize later.

04 Game Design

4.1 Core loop

Player auto-runs left to right (visually, the world scrolls right to left toward a stationary player). Obstacles spawn ahead and move toward the player. Player jumps to avoid them. Surviving longer increases both speed and score. One collision ends the run.

4.2 Controls

Key	Action
Space / ↑	Jump
q / Ctrl+C	Quit
r	Restart (only on game-over screen)

Exact bindings live in config (see [Data Model](#)) so they're changeable without touching code.

4.3 Player

- Fixed horizontal position on screen; only vertical movement (jump).
- Jump is a fixed-height arc: triggered by one key press, cannot be extended mid-air (no "hold to jump higher" in v1 — keeps input handling simple and predictable).
- Cannot jump again until landed (no double-jump in v1).

4.4 Obstacles — v1

- **Single type:** one ground-level obstacle (e.g. a cactus-style block), per your scope decision. Additional types are a v2 item (see [Roadmap](#)) and are designed for from the start (obstacles are a list of typed entities, not a hardcoded single object) so adding a second type later doesn't require restructuring.
- Spawn at a random-but-bounded interval so gaps are never impossible to clear.

4.5 Difficulty curve

- Game speed starts at a fixed base value and increases smoothly with elapsed survival time (not in sudden steps), handled by `engine/difficulty.py`.
- Obstacle spawn interval scales down slightly as speed increases, keeping challenge consistent rather than just "faster but same gaps."
- Exact curve (formula/constants) is a tunable value, not fixed in this doc — expect it to be adjusted by playtesting during implementation.

4.6 Scoring

- Score increases automatically over time survived (not from any explicit player action).
- On game over, score is compared to the stored high score; the higher value is kept.

4.7 Screens

- **Start (MENU):** game title, high score, "press any key to start."
- **Playing:** the runner, current score shown live.
- **Game Over:** final score, high score (with a "new high score!" indicator if beaten), restart/quit prompt.

4.8 What's deliberately left out of v1

Duck/crouch, multiple obstacle types, sound, and themes are all real, natural extensions — they're deferred so v1 stays a small, complete, well-tested loop rather than a half-finished larger scope. See [Roadmap](#).

05 Terminal Behavior

5.1 Why `curses`

`curses` is the standard-library terminal UI toolkit on macOS and Linux, gives direct control over cursor position and screen buffering, and handles a lot of terminal-capability differences internally. It's the natural choice for "no external UI dependency" — no ncurses-alternative package needed.

5.2 Startup / teardown

`ui/terminal.py` wraps `curses.wrapper()` (or an equivalent explicit `init/try/finally`), which guarantees:

- Raw/cbreak mode and non-blocking input are set up on start.
- The terminal is always restored to its normal (cooked) state on exit — including on a crash or `Ctrl+C` — so the user's shell is never left in a broken state. This directly satisfies NFR-2 in the requirements.

5.3 Shell independence (Bash vs Zsh)

TINO doesn't do anything shell-specific — the game itself is a compiled/installed Python entry point (`tino`), not a shell script, so it behaves identically regardless of which shell launched it. The only shell-touching pieces are the dev-only scripts in `scripts/` (`setup/lint/test`), which are plain POSIX-compatible Bash and don't affect the shipped game.

5.4 Terminal size / resize

- On start, TINO reads the current terminal dimensions and lays out the play area to fit.
- If the terminal is resized mid-run, `ui/terminal.py` detects it (via `curses`' resize signal handling) and the renderer adapts on the next frame rather than crashing or drawing out of bounds.
- If the terminal is too small to render a playable area, TINO shows a clear message asking the user to resize, rather than rendering garbled output.

5.5 Input handling

- Input is read non-blocking each frame (`ui/input.py`), so a held or absent keypress never stalls the game loop.
- Keycodes are mapped to abstract actions (`JUMP`, `QUIT`, `RESTART`) at this layer — the engine only ever sees actions, never raw keys, which is what keeps the engine terminal-agnostic (see [Architecture §3.1](#)).

5.6 macOS vs Linux differences

Both are POSIX terminals with curses available in the Python standard library, so no OS-specific branching is expected. The only practical difference to watch for during implementation is default terminal emulator quirks (e.g. macOS Terminal.app vs iTerm2 vs common Linux emulators like GNOME Terminal) around color support and key-escape-sequence timing — handled generically by curses rather than special-cased per OS.

06 CLI

Built with `argparse`, `tinot` with no arguments launches the game directly (FR-1) — subcommands are for everything else.

Command	Effect
<code>tinot</code>	Launch the game (default behavior, no subcommand needed).
<code>tinot score</code>	Print the current stored high score and exit — no terminal takeover.
<code>tinot reset-score</code>	Clear the stored high score (with a confirmation prompt).
<code>tinot --version</code>	Print the installed version.
<code>tinot --help</code>	Standard argparse help output.

Design notes

- Launching straight into the game with zero arguments (rather than requiring `tinot play`) matches how the project is meant to be used — a quick terminal diversion, not a tool with a formal command structure.
- `score` and `reset-score` are deliberately simple, non-interactive-terminal commands (plain stdout), so they work in scripts/pipelines too.
- All CLI parsing lives in `cli.py` and only decides *what to run* — it never contains game logic itself (see [Architecture §3.3](#)).

07 Data Model

7.1 Storage location

Resolved by `storage/paths.py` using a standard per-user data directory (XDG-style on Linux, `~/Library/Application Support` on macOS), via `pathlib`. Not hardcoded to the current working directory, so `tino` works correctly regardless of where it's launched from.

7.2 High score file

```
{
  "high_score": 0,
  "updated_at": "2026-09-15T12:00:00Z"
}
```

- Read/written exclusively through `storage/highscore.py`'s `HighScoreStore` interface (see [Architecture §3.7](#)) — nothing else touches this file directly.
- A missing file is treated as `high_score: 0`. A corrupt/unreadable file is treated the same way rather than crashing the app (NFR-2).

7.3 Config file (optional)

```
{
  "keybindings": {
    "jump": "space",
    "quit": "q",
    "restart": "r"
  }
}
```

- Entirely optional — if absent, `config.py` uses in-code defaults. This keeps FR (zero required setup) true while still allowing customization.
- Only recognized keys are read; unknown keys are ignored rather than causing an error, so the file can be extended later without breaking older installs.

7.4 Forward-compatibility note

Both files are plain flat JSON on purpose — no versioning field is added in v1 because there's nothing yet to version against. If the schema needs to change later (e.g. once cloud sync is added), a `schema_version` field can be introduced at that point without breaking existing local files (missing version treated as `1`).

08 Testing

8.1 Strategy

Because engine logic is deliberately isolated from curses (see [Architecture §3.1](#)), the majority of the test suite runs with no real terminal attached — plain `pytest`, fast, and CI-friendly.

Layer	How it's tested
<code>engine/</code> <code>collision.py</code>	Pure unit tests — feed known positions, assert overlap/no-overlap.
<code>engine/entities.py</code>	Unit tests on jump trajectory math (position over successive ticks).
<code>engine/</code> <code>difficulty.py</code>	Unit tests asserting speed/spawn-rate as a function of elapsed time.
<code>engine/state.py</code> / <code>game.py</code>	Unit tests driving the state machine through legal and illegal transitions.
<code>storage/</code> <code>highscore.py</code>	Unit tests against a temp directory (via <code>tmp_path</code> fixture) — including missing-file and corrupt-file cases.
<code>config.py</code>	Unit tests for default merging and unknown-key handling.
<code>ui/*</code>	Thin integration checks only (e.g. renderer produces expected output for a given state) — not exhaustively unit tested, since it's a thin, mostly side-effecting layer.

8.2 Structure

```
tests/
├─ unit/
│   ├── test_collision.py
│   ├── test_entities.py
│   ├── test_difficulty.py
│   ├── test_state.py
│   ├── test_highscore.py
│   └─ test_config.py
└─ integration/
    └─ test_app_wiring.py  # confirms app.py assembles engine + ui + storage correctly
```

8.3 Tooling

- **pytest** for the test runner.
- **Ruff** for linting and formatting (run in CI and via `scripts/lint.sh` locally).

8.4 CI

GitHub Actions (`.github/workflows/ci.yml`) runs on every push and PR:

1. Set up Python 3.12+.
2. Install dependencies.
3. Run Ruff (lint + format check).
4. Run the full pytest suite.

A failing lint or test check blocks merge.

09 Tech Stack

Only what's actually used in TINO, and why each was chosen.

Tool	Used for	Why this, specifically
Python 3.12+	The entire application	Standard library alone covers everything needed (terminal UI, JSON, paths) — no external runtime required, matching the "no GUI framework/runtime" goal in the overview.
curses	Terminal rendering & raw input	Standard-library terminal toolkit on macOS/Linux; handles low-level terminal capability differences without an extra dependency.
argparse	CLI (<code>тино</code> , <code>тино score</code> , <code>тино reset-score</code>)	Standard library, sufficient for a small, fixed set of commands — no need for a heavier CLI framework.
JSON + pathlib	Local persistence (high score, config)	Human-readable, no schema/migration tooling needed at this scale; <code>pathlib</code> gives cross-platform (macOS/Linux) file path handling.
pytest	Testing	De facto standard Python test runner; fixtures (e.g. <code>tmp_path</code>) are a natural fit for testing the storage layer in isolation.
Ruff	Linting + formatting	Single fast tool covering both lint and format, replacing the need for separate linter + formatter + import-sorter.
Git + GitHub	Version control, collaboration	Standard choice; also hosts CI and (later) releases.
GitHub Actions	CI	Runs lint + tests on every push, native to GitHub, no separate CI service needed.
pyproject.toml	Project/packaging configuration	Modern standard single-file config for build metadata, dependencies, and tool settings (Ruff, pytest) — avoids scattering config across multiple files.
Bash	Dev/setup scripts only (<code>scripts/</code>)	Convenience wrappers for contributors (setup/lint/test) — not part of the shipped game, so it doesn't affect shell independence (see Terminal Behavior §5.3).

Deliberately not used (for now): any third-party UI/game framework, a database, a web framework, or a packaging step beyond `pyproject.toml` — none are justified by v1's actual scope. PyPI publishing is planned (see [Roadmap](#)) but isn't a v1 requirement.

10 Roadmap

v1 — Core game (current scope)

- Single obstacle type, jump-only mechanic, progressive difficulty.
- Local JSON high score, optional local config.
- `tino` / `tino score` / `tino reset-score` CLI.
- Full unit test coverage on engine + storage, CI on every push.
- Install via clone + run locally.

v2 — Depth & polish

- Additional obstacle types with varied spawn patterns.
- Duck/crouch mechanic (for overhead obstacles).
- Visual polish (color, simple animation frames within curses' capabilities).
- Config-driven key rebinding exposed to the player (not just editable JSON).

v3 — Distribution & reach

- Publish to PyPI (`pip install tino`) — the packaging groundwork (`pyproject.toml`) is already in place from v1, so this is primarily a release-process step, not a rearchitecture.
- Optional cloud/remote high-score sync, implemented as a second `HighScoreStore` implementation (see [Architecture §3.7](#)) — no engine changes required.
- Evaluate Windows support if there's demand (would require revisiting the curses dependency, likely via `windows-curses`).

Explicitly not planned

- Networked multiplayer.
- A GUI/web version — if that's ever wanted, it's a different project sharing the engine, not a pivot of TINO itself.